Министерство образования Республики Беларусь

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №3
«Программирование системного таймера»
Вариант 10

Выполнил:
Студент группы 450502
Новицкий Е.Ю.

Проверил:
Преподаватель
Марзалюк А.В.

Минск, 2026

1. Постановка задачи

Запрограммировать второй канал таймера таким образом, чтобы динамик компьютера издавал звуки.

Для всех каналов таймера считать слово состояния и вывести его на экран в двоичной форме.

2. Алгоритм

Для того чтобы динамик компьютера издавал звуки, необходимо выполнить следующие действия:

- Вывести в порт управляющего регистра с адресом 43h управляющее слово 10110110, соответствующее каналу 2, режиму 3
- Установить значение счётчика канала 2 таймера: в порт 42h вывести значение, полученное при разделении 1193180 на требуемую частоту в герцах, причём вначале вывести младший, а затем старший байты.
- Установить в 1 два младших бита порта 61h для включения звука. Для этого вначале считывается байт из порта 61h в рабочую ячейку памяти, устанавливаются нужные биты, затем выводится новое значение байта в порт 61h.
- Установить в 0 два младших бита порта 61h для выключения звука.

Для чтения слова состояния каналов необходимо:

- Вывести в порт управляющего регистра с адресом 43h управляющее слово, соответствующее команде RBC (*Чтение состояния канала)* и номеру канала.
- Вывести из порта нужного канала слово состояния.

3. Листинг программы

Далее приведен листинг программы, реализующей все поставленные задачи.

```
#include <stdio.h>
#include <dos.h>
#include <conio.h>

/* base frequency of system timer */
#define PIT_BASE_FREQUENCY 1193180UL

/* ports of system timer and system dynamic */
#define PIT_CHANNEL0_PORT 0x40
#define PIT_CHANNEL1_PORT 0x41
#define PIT_CHANNEL2_PORT 0x42
#define PIT_CONTROL_PORT  0x43
#define SPEAKER_PORT      0x61

typedef struct
```

```
{
    unsigned int frequency;
    unsigned int duration;
    unsigned int pause;
} Note;


void setSpeakerFrequency(unsigned int frequency)
{
    unsigned int divider;
    unsigned char lowByte;
    unsigned char highByte;

    if (frequency == 0)
    {
        return;
    }

    divider = (unsigned int)(PIT_BASE_FREQUENCY / frequency);

    lowByte = (unsigned char)(divider & 0xFF);
    highByte = (unsigned char)((divider >> 8) & 0xFF);

    outp(PIT_CONTROL_PORT, 0xB6);

    outp(PIT_CHANNEL2_PORT, lowByte);
    outp(PIT_CHANNEL2_PORT, highByte);

}

void speakerOn(void)
{
    unsigned char value;

    value = inp(SPEAKER_PORT);

    value = value | 0x03;

    outp(SPEAKER_PORT, value);
}


void speakerOff(void)
{
    unsigned char value;

    value = inp(SPEAKER_PORT);
```

```
        value = value & 0xFC;

        outp(SPEAKER_PORT, value);
    }


    void playNote(unsigned int frequency, unsigned int duration, un-
signed int pause)
    {
        if (frequency == 0)
        {
            delay(duration);
            delay(pause);
            return;
        }

        setSpeakerFrequency(frequency);
        speakerOn();
        delay(duration);
        speakerOff();
        delay(pause);

    }

    void playTestMelody(void)
    {
        Note melody[] =
            {
            {440, 300, 100},
            {494, 300, 100},
            {523, 300, 100},
            {587, 300, 100},
            {659, 500, 150}
        };

        int i;
        int count = sizeof(melody) / sizeof(melody[0]);

        printf("Playing test melody...\n");

        for (i = 0; i < count; i++)
        {
            playNote(melody[i].frequency, melody[i].duration, mel-
ody[i].pause);
        }

    }
```

```
void playMasterOfPuppets(void)
{
    Note melody[] =
    {

        {329, 180, 40},
        {329, 180, 40},
        {329, 180, 40},
        {311, 180, 40},
        {329, 180, 40},
        {392, 250, 60},

        {293, 180, 40},
        {293, 180, 40},
        {293, 180, 40},
        {277, 180, 40},
        {293, 180, 40},
        {349, 250, 60},

        {329, 180, 40},
        {329, 180, 40},
        {392, 250, 60},
        {440, 300, 80},

        {0,   200, 0},    /* pause */

        {392, 180, 40},
        {349, 180, 40},
        {329, 250, 60},
        {293, 300, 80}

    };

    int i;
    int count = sizeof(melody) / sizeof(melody[0]);

    printf("Playing: Metallica - Master of Puppets\n");

    for (i = 0; i < count; i++)
    {
        playNote(melody[i].frequency, melody[i].duration, mel-
ody[i].pause);
    }
}
```

```
void printBinaryByte(unsigned char value)
{
    int i;

    for(i = 7; i >= 0; i--)
    {
        if(value & (1 << i))
        {
            printf("1");
        }
        else
        {
            printf("0");
        }
    }
}



void printStatusWords(void)
{
    unsigned char status;
    int ports[3] = {PIT_CHANNEL0_PORT, PIT_CHANNEL1_PORT,
PIT_CHANNEL2_PORT};

    unsigned char controlWords[3] = {0xE2, 0xE4, 0xE8};

    int channel;

    printf("\nStatus words of timer channels:\n");

    for (channel = 0; channel < 3; channel++)
    {
        outp(PIT_CONTROL_PORT, controlWords[channel]);

        status = inp(ports[channel]);

        printf("Channel %d: ", channel);
        printBinaryByte(status);
        printf("\n");
    }

}

unsigned int readCounterValue(int channelPort, unsigned char
latchCommand)
{
    unsigned char lowByte;
```

```
        unsigned char highByte;
        unsigned int value;

        outp(PIT_CONTROL_PORT, latchCommand);

        lowByte = inp(channelPort);
        highByte = inp(channelPort);

        value = ((unsigned int) highByte << 8) | lowByte;

        return value;

    }


    void printCounterValues(void)
    {
        int ports[3] = {PIT_CHANNEL0_PORT, PIT_CHANNEL1_PORT,
PIT_CHANNEL2_PORT};

        unsigned char latchCommands[3] = {0x00, 0x40, 0x80};

        unsigned int counterValue;
        int channel;

        printf("\nCounter values (CE) in hexadecimal:\n");

        for (channel = 0; channel < 3; channel++)
        {
            counterValue = readCounterValue(ports[channel], latchCom-
mands[channel]);
            printf("Channel %d: %04Xh\n", channel, counterValue);

        }

    }



    int showMenu(void)
    {
        int option;

        printf("\n==============================\n");
        printf("Programmable Interval Timer Lab\n");
        printf("==============================\n");
        printf("1. Play test melody\n");
        printf("2. Play Metallica - Master of Puppets\n");
```

```
    printf("3. Show status words\n");
    printf("4. Show counter values (CE)\n");
    printf("5. Exit\n");
    printf("Choose option: ");

    scanf("%d", &option);

    return option;
}


int main(void)
{
    int option;

    clrscr();

    while (1)
    {
        option = showMenu();

        switch (option)
        {
            case 1:
                clrscr();
                playTestMelody();
                break;

            case 2:
                clrscr();
                playMasterOfPuppets();
                break;

            case 3:
                clrscr();
                printStatusWords();
                break;

            case 4:
                clrscr();
                printCounterValues();
                break;

            case 5:
                clrscr();
                speakerOff();
                return 0;
```

```
                default:
                    clrscr();
                    printf("Invalid option. Try again.\n");
                    break;
            }
        }
    }
```

4. Тестирование программы

Во время работы программы происходит звучание системного динамика. Также для всех каналов таймера выводится на экран в двоичной форме слово состояния:

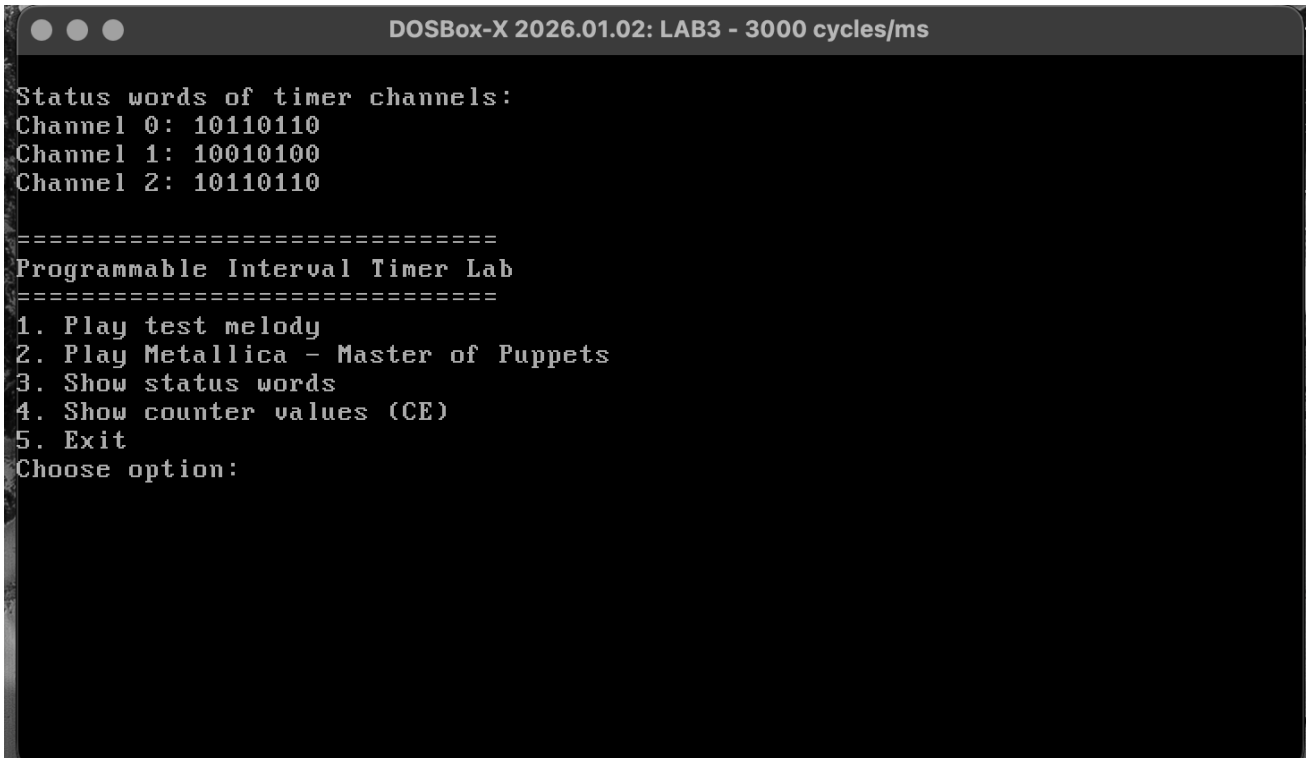

Рисунок 4.1 – Результат работы программы при выводе слов состояния каналов таймера.

5. Заключение

В ходе лабораторной работы удалось запрограммировать второй канал таймера таким образом, чтобы динамик компьютера издавал звук, а также для всех каналов таймера было считано слово состояния и выведено на экран в двоичной форме.

Программа компилировалась в Turbo C++ и запускалась в DOS, который эмулировался с помощью DosBox-X.